Repository-Level Code Indexing and Token Optimization for LLM Agents

The Architectural and Economic Imperative of Context Optimization

The integration of Large Language Models (LLMs) into automated software engineering has precipitated a paradigm shift from deterministic, rule-based static analysis to probabilistic, reasoning-driven artificial intelligence. In their earliest iterations, code LLMs were largely restricted to function-level or file-level code generation, excelling at predicting localized syntax or identifying isolated errors within a highly constrained context window.[1] However, modern enterprise software development is inherently decentralized and deeply interconnected. It involves intricate networks of cross-file dependencies, expansive architectural hierarchies, and domain-specific semantic frameworks that span hundreds of thousands of lines of code. This reality has necessitated the evolution of Repository-Level Code Generation (RLCG), a domain where models must reason across an entire codebase, capturing long-range dependencies, ensuring global semantic consistency, and generating coherent logic that spans multiple discrete files.[1]

As models scale to process entire repositories, they encounter a severe architectural and economic bottleneck: the limitation and cost of the context window. In the computational architecture of an LLM, the model weights function as the central processing unit, while the context window acts analogously to random-access memory (RAM).[3] Just as a traditional operating system must meticulously curate which processes and data streams reside in physical RAM to prevent system degradation, an LLM agent must be supplied with highly curated, mathematically optimized context.[3] Feeding a massive, multi-megabyte software repository directly into the context window of an LLM introduces profound computational and financial inefficiencies.

Commercial LLM providers calculate financial costs based precisely on the volume of input and output tokens processed during inference.[4] Consequently, transmitting thousands of lines of irrelevant, unoptimized source code for a relatively simple query results in exorbitant financial expenditures, high computational latency, and significant noise overload.[4] The economic reality of scaling these agents across enterprise environments means that token usage optimization is not merely a supplementary engineering goal; it is a fundamental prerequisite for deployment.

Furthermore, simply scaling the context window to accommodate millions of tokens—a trend observed in models from late 2024 through early 2026—does not resolve the underlying systemic vulnerabilities. As context length increases exponentially, LLMs inevitably suffer from "context pollution." In this state, irrelevant or redundant data dilutes the transformer's attention mechanism, leading to severe performance degradation, the hallucination of non-existent

application programming interfaces (APIs), and a documented failure to locate critical information buried in the middle of the prompt.[6] For long-horizon tasks—such as comprehensive codebase migrations, multi-file refactoring, or deep security audits spanning several hours of autonomous work—agents require specialized techniques to maintain coherence over sequences of actions where the aggregate token count vastly exceeds the LLM's absolute context limit.[7]

To build scalable, accurate, and cost-effective LLM coding agents, the software engineering research community has decisively shifted focus. Rather than merely relying on raw context window expansion, the current state of the art emphasizes engineering intelligent, highly optimized retrieval and indexing systems. The objective is to provide the LLM with the exact necessary information at the precise moment it is required, optimizing token usage while simultaneously maximizing repository-level comprehension. Peer-reviewed research from top-tier computational linguistics and software engineering venues—including the International Conference on Machine Learning (ICML), the Association for Computational Linguistics (ACL), Empirical Methods in Natural Language Processing (EMNLP), and the Automated Software Engineering (ASE) conference—highlights a rapid evolution in these methodologies.[4] This report synthesizes the proven, peer-reviewed indexing methodologies, retrieval policies, and state-management frameworks defining the 2026 landscape of LLM code optimization.

Multi-Agent Routing and the Economics of Prompt Compression

Before examining specific indexing algorithms, it is essential to understand the structural workflow optimizations that precede retrieval. In sophisticated LLM coding environments, token optimization begins with the architectural distribution of cognitive load. Large foundational models inherently waste substantial computational resources by loading exhaustive contextual data into memory, even when an agent requires only a minute fraction of that information to execute a sub-task.[10]

To sustain coherence and minimize token expenditure over extended operations, architectures have transitioned from "flat" context passing—where all potentially relevant retrieved data is appended blindly to a monolithic prompt—to dynamic, multi-agent compute protocols (MCP).[10] Empirical deployments of these protocols have demonstrated that dividing complex repository tasks among highly specialized, narrow-scope agents yields dramatic efficiency gains. In a documented multi-agent retrieval configuration, the workload was systematically partitioned across three distinct agent topologies: an extractor agent tasked solely with indexing, an analyzer agent for dependency mapping, and an answerer agent for final code generation.[10]

Through a dedicated routing layer, each agent received only the specific, narrowly tailored information strictly relevant to its immediate task.[10] This protocol eliminated the redundant passing of full context histories between turns. The implementation of this agentic retrieval protocol resulted in a 60% reduction in aggregate token usage compared to the baseline flat

RAG-style context passing.[10] Crucially, this massive reduction in prompt size yielded a corresponding 35% improvement in inference latency, as the underlying models processed significantly fewer input tokens per API call.[10]

Counterintuitively, the accuracy of the generated code did not degrade despite the 60% reduction in provided context. In fact, researchers documented slight gains in accuracy, attributing the improvement to cleaner, more tightly focused prompts that prevented the LLM's attention mechanism from being derailed by spurious correlations or noisy, irrelevant code blocks.[10] This principle of aggressive context partitioning is further supported by industry best practices, which advocate for exposing context dynamically via tool calls rather than stuffing instructions into the system prompt. By partitioning important details and allowing the LLM to fetch relevant context strictly on demand, agents avoid upfront token saturation.[10]

This routing methodology is increasingly pushed to the local edge. Proposed architectures for modern Integrated Development Environments (IDEs) advocate for a "Local Filter and Smart Caching" paradigm to intercept and optimize queries before they reach expensive frontier models.[5] In this workflow, an AST parser or a lightweight local model scans the repository upon initialization, extracting function names, line numbers, file paths, and docstrings, storing them in a local SQLite database or lightweight vector store.[5] When a developer issues a command, the local model acts as a pre-filter, querying the local cache to build a minimal, highly enriched prompt.[5] By executing the indexing and initial retrieval locally, the system ensures the expensive cloud LLM receives a purely focused context, radically reducing token expenditure while building an internal graph of function interactions to further narrow future search spaces.[5]

Structure-Aware Code Indexing: AST-Based Partitioning

The foundation of any LLM-based repository agent is its retrieval index. Traditional Retrieval-Augmented Generation (RAG) pipelines, originally designed for natural language processing, rely heavily on rudimentary text-chunking heuristics. These legacy systems divide documents into fixed-size, line-based segments or rely on overlapping character counts.[12] While this naive approach is generally acceptable for unstructured prose, it is profoundly destructive when applied to source code.

Source code is not unstructured text; it possesses rigid inherent syntactic boundaries, logical hierarchies, and strict scoping rules.[12] A fixed-size chunking algorithm operates blindly regarding these boundaries. Consequently, it will frequently split a function definition away from its executing body, separate a class method from its instantiated variables, or isolate a crucial return statement from the logical premise that triggered it.[12] When an LLM retrieves such a syntactically fragmented chunk from a vector database, it loses vital structural context. This fragmentation leads the model to hallucinate incorrect assumptions about variable scopes, parameter types, and return values, culminating in catastrophic downstream code generation

failures.[12]

The cAST Methodology (Chunking via Abstract Syntax Trees)

To rectify the severe limitations of fixed-size text chunking, researchers have pioneered structure-aware indexing methodologies, most prominently the cAST (Chunking via Abstract Syntax Trees) framework, which has been rigorously evaluated in peer-reviewed literature.[12] An Abstract Syntax Tree is a computational representation of source code as a hierarchical, directed tree structure. Every node within the AST corresponds to a fundamental, typed programmatic unit—such as a variable declaration, a while loop, an if condition, or a class instantiation.[12]

The cAST methodology overhauls the indexing preprocessing pipeline by parsing the raw repository code into complete ASTs using dedicated grammar tools, such as ANTLR (ANother Tool for Language Recognition).[12] Once the code is structured as a tree, the algorithm applies a recursive, split-then-merge process.[12] Rather than severing code based on arbitrary token counts, cAST recursively breaks down massive AST root nodes into progressively smaller, yet syntactically complete, subtrees.[9]

Crucially, the algorithm includes a merging phase where sibling nodes within the tree are recombined, provided that the resulting merged block respects the maximum token embedding limits of the underlying vectorizer.[12] This structure-aware orchestration ensures that every chunk indexed in the database constitutes a self-contained, semantically coherent unit of code, completely devoid of dangling syntactical fragments.[13] Furthermore, these AST representations capture deep semantic layer definitions, control flow patterns, and command relationships, providing a vastly superior contextual map for the LLM to traverse.[16]

Empirical Performance of AST Chunking

The integration of cAST and similar syntactic partitioning algorithms into LLM agent pipelines dramatically improves the integrity of the retrieved context. Because the code chunks rigidly respect semantic boundaries, the LLM receives unbroken logic flows, allowing it to accurately identify return values and integrate them flawlessly into the target codebase.[12]

Extensive evaluations in peer-reviewed environments demonstrate that this recursive AST partitioning significantly outpaces traditional line-based chunking across diverse programming languages and generation tasks. On the rigorous RepoEval benchmark, the implementation of structure-aware AST chunking boosted the Recall@5 metric by 4.3 points.[13] Furthermore, on the highly demanding SWE-bench generation task—which requires agents to autonomously resolve real-world GitHub issues—AST chunking improved the Pass@1 generation success rate by 2.67 points.[13]

Industrial applications corroborate these academic findings. Engineering teams deploying systems akin to the Databricks Knowledge Assistant systematically compared fixed-size baseline chunking against structure-aware AST chunking using the MLflow GenAI evaluation

framework.[14] Their empirical evaluations confirmed that failing to index code via its syntactic components severely degraded the LLM judge's retrieval scores, proving that structural integrity is paramount for complex, cross-file reasoning tasks.[14] The resulting CodeHierarchyKeywordQueryEngine allows the LLM to comprehend the exact hierarchy of a repository, significantly reducing the token waste associated with indexing and retrieving broken code.[18]

Metric / Feature	Legacy Fixed-Size Chunking	cAST (AST-Based Chunking)
Boundary Recognition	Blind token/line count.	Strict semantic & syntactic boundaries.
Code Integrity	High probability of split logic and dangling syntax.	100% syntactically valid and self-contained subtrees.
RepoEval Recall@5	Baseline	+4.3 points improvement.[13]
SWE-bench Pass@1	Baseline	+2.67 points improvement.[13]
Token Efficiency	Low (retrieves useless, uncompilable fragments).	High (retrieves only coherent, executable logic blocks).

Bridging the Domain Gap via Hierarchical Summarization

While AST chunking optimizes the extraction of precise programmatic logic, LLM agents frequently encounter a secondary retrieval bottleneck: the domain gap. When dealing with bug reports generated by non-technical end-users, or high-level natural language queries from product managers, a severe vocabulary mismatch occurs.[19] Standard similarity-based vector retrieval fails in these scenarios because the natural language embeddings of a query like "the checkout cart is dropping items" do not map cleanly to the highly specific mathematical embeddings of a developer's cart_state_mutation_handler function.[19]

Attempting to solve this by dumping the entire repository into the LLM and asking it to find the correlation results in immediate context window overflow. To bridge this semantic gap without incurring astronomical token costs, researchers have developed structure-aware hierarchical summarization methodologies.[19]

This approach completely restructures the LLM's initial view of the repository. During the pre-processing phase, the system constructs an abstract repository tree.[19] It then utilizes a highly efficient, primed LLM to generate natural language summaries at three distinct, escalating levels of granularity: the global project level, the directory level, and the individual file level.[19] Consequently, the expansive repository is compressed into a reduced-size, tree-structured prompt that preserves essential architectural and functional information while completely mitigating the context window bottleneck.[19]

During the inference phase, the LLM agent employs a task-agnostic, top-down search strategy.[19] Confronted with an ambiguous bug report, the agent first evaluates the directory-level summaries to deduce which broad module is conceptually responsible for the failing behavior.[19] Once isolated, the agent navigates down to the file-level summaries within that specific directory, progressively narrowing the search space until it pinpoints the exact file.[19] Only at this terminal stage does the agent retrieve the raw AST code chunks.

Evaluated on a real-world dataset of complex Jira issues extracted from a large-scale industrial project, this top-down hierarchical summarization approach achieved remarkable precision. It significantly outperformed both standard Flat Retrieval baselines and state-of-the-art RAG systems, achieving a Pass@10 score of 0.89 and a Recall@10 score of 0.33.[19] By empowering the LLM to translate non-technical intent into architectural navigation, this methodology eliminates the need to index massive code blocks for initial discovery, thereby drastically reducing token expenditure during the bug localization phase.[19]

The Economics and Logic of Selective Retrieval Policies

The prevailing assumption in early AI coding architectures was that providing more context invariably leads to superior outputs. However, extensive empirical analysis presented at ICML and other top-tier venues reveals that invariant, ubiquitous retrieval exposes severe efficiency and robustness flaws.[20] In the context of repository-level code completion, supplying retrieved cross-file context when it is not strictly necessary introduces distracting variables, conflicting API versions, and irrelevant boilerplate, which statistically degrades the LLM's output quality.[20]

The Repoformer Paradigm and Self-Supervised Abstention

A groundbreaking study introduced the Repoformer framework, fundamentally challenging the "always-retrieve" RAG paradigm. The researchers investigated the efficacy of RAG during routine code completion tasks and uncovered a staggering metric: approximately 80% of the time, external context retrieval does not improve the quality of the generated code.[21] Modern LLMs pretrained on vast source code corpora are already remarkably proficient at completing arbitrary functions based solely on the immediate, local, in-file context.[21] Therefore, invoking a computationally expensive retrieval process, executing a vector search, and appending the results into the prompt for every single autocomplete request wastes massive amounts of

compute time and exacts a heavy toll on the token budget.[21]

Repoformer resolves this inefficiency by introducing a self-supervised "Selective Retrieval" policy model. Instead of relying on a hardcoded, rule-based heuristic, a code LLM is fine-tuned to act as an autonomous decision-making agent that evaluates whether external repository context is genuinely required.[21] If the model determines that the local context is insufficient—typically because the completion relies on an unimported API or a complex cross-file dependency—it emits a specialized <retrieve> token, triggering the RAG pipeline.[21] Conversely, if it calculates it can complete the code autonomously, it emits a <no_retrieve> token, entirely bypassing the retrieval infrastructure.[20]

The dataset required to train this highly accurate selective policy is generated autonomously, requiring zero human annotation.[21] The construction process involves sampling code from open-license repositories and randomly masking out specific lines.[21] The system then retrieves related code from elsewhere in the repository using Jaccard similarity—a highly efficient metric calculating the intersection over union of symbol sequences, which researchers found outperformed both semantic embeddings (UniXCoder) and syntax-tree methods (CodeBLEU) for this specific pre-filtering step.[21] An LLM then attempts to reconstruct the masked code twice: once with the retrieved context, and once without.[21] The training examples are automatically labeled based on whether the retrieval actually yielded a mathematically superior generation.[21]

During fine-tuning, the model utilizes a "Fill-In-The-Middle" (FIM) mechanism. The masked target code is excised, and the preceding and succeeding contextual sections are demarcated with special tokens, allowing the model to analyze the full surrounding structure before making its retrieval decision.[21] The policy model operates under adjustable settings, such as Selective_G (a greedy approach that retrieves if the most likely next token originates outside the file) or Selective_T (a threshold approach requiring the probability of retrieval helpfulness to exceed a strict mathematical bound).[21]

Performance Metrics of Selective Retrieval

The performance gains achieved by selective retrieval are profound and fundamentally alter the economics of LLM deployment. The fine-tuned Repoformer model achieves over 85% accuracy in its retrieval decision-making, learning to make highly accurate abstention judgments.[21]

By actively avoiding unnecessary searches and context stuffing, the selective framework accelerates online inference speeds by up to 70% compared to models that enforce ubiquitous retrieval.[20] This speedup directly translates to a massive reduction in the computational load and financial cost associated with processing unnecessary context tokens.[21] Furthermore, this selective approach demonstrably increases the model's structural robustness. By restricting retrieval only to scenarios where it provides high-value signal, Repoformer significantly reduces "harmful retrievals"—instances where noisy or conflicting external context confuses the

attention mechanism and degrades the output.[20]

Operating as a plug-and-play policy model, Repoformer can be deployed as a lightweight gatekeeper in front of massive generation models, securing state-of-the-art repository-level code completion performance on stringent benchmarks including RepoEval, CrossCodeEval, and CrossCodeLongEval, while simultaneously halving the token expenditure.[20]

Revisiting Lexical Search: Index-Free Retrieval Paradigms

Simultaneous to the development of selective retrieval policies, the academic software engineering community initiated a rigorous re-evaluation regarding the absolute necessity of dense semantic indexing. Traditional RAG systems are predicated on converting source code into high-dimensional vector embeddings, storing them in specialized vector databases, and executing approximate nearest-neighbor mathematical searches.[23] While mathematically elegant, this semantic indexing infrastructure incurs immense computational overhead during both the initial index construction and continuous maintenance, as the vectors must be recomputed and updated every time a developer commits a change to a file.[23]

GrepRAG and the Superiority of Precise Lexical Matching

Motivated by how human engineers empirically navigate complex codebases—relying heavily on ultra-fast, lightweight terminal utilities rather than semantic search engines—researchers developed GrepRAG.[23] This framework systematically investigates the efficacy of lightweight, index-free lexical retrieval for repository-level comprehension.[23]

The core premise underlying GrepRAG is that software logic relies on exact syntax and highly specific, deterministic identifiers. While semantic vector search excels at finding code that conceptually aligns with a natural language query (e.g., "function that prints to screen"), it frequently struggles and hallucinates when required to locate the exact, rigid definition of a highly specific custom entity, such as auth_token_v2_middleware_factory.[23] Lexical search utilities, conversely, locate exact character string matches instantaneously.

The foundational implementation, termed Naive GrepRAG, grants the LLM agent the autonomy to generate standard ripgrep terminal commands to search the target repository.[23] Despite lacking any semantic understanding, deep neural embeddings, or pre-computed vector indexes, this astonishingly simple index-free approach achieves performance highly comparable to sophisticated, computationally heavy graph-based RAG baselines.[23] Analysis reveals that the effectiveness of this naive approach stems from its ability to retrieve lexically precise code fragments that are spatially closer to the completion site, avoiding the semantic drift inherent in vector space.[23]

Overcoming Lexical Limitations via Post-Processing

However, pure index-free lexical retrieval suffers from documented limitations. Chief among

these is its extreme sensitivity to high-frequency, ambiguous keywords; querying a common term like data or config will return thousands of useless matches, instantly overflowing the context window.[23] Additionally, lexical search suffers from context fragmentation caused by rigid, line-based truncation boundaries surrounding the grep match.[23]

To neutralize these issues while preserving the massive computational cost savings of the index-free advantage, the complete GrepRAG framework implements a highly optimized, lightweight post-processing pipeline.[23] Following the initial lexical string pull, the system applies an identifier-weighted re-ranking algorithm to elevate the most critical variables, combined with a structure-aware deduplication process to purge redundant matches before appending the context to the LLM.[23]

Extensive empirical evaluations across the CrossCodeEval and RepoEval-Updated datasets definitively demonstrate the power of this optimized lexical approach. GrepRAG consistently outperforms state-of-the-art semantic and graph-based indexing methods, achieving a remarkable 7.04% to 15.58% relative improvement in exact code match (EM) metrics over the best existing baseline on CrossCodeEval.[23] By demonstrating that simple, highly optimized grep-like retrieval can surpass complex embedding models, GrepRAG proves that for specific LLM coding agents, bypassing index construction entirely reduces infrastructure costs and token usage while simultaneously enhancing repository-level precision.[24]

Advanced Multi-Path and Aligned Retrieval Architectures

When repository queries exhibit extreme complexity—where neither selective abstention nor index-free lexical search provides sufficient context—agents must deploy sophisticated, multi-dimensional retrieval pipelines. However, legacy RAG systems frequently succumb to two critical vulnerabilities: query misalignment (where the natural language query fundamentally fails to match the structural nomenclature of the target code) and information redundancy (where the retriever pulls the same overlapping boilerplate functions repeatedly, squandering the token limit).[25] To counteract these systemic flaws, advanced frameworks presented at EMNLP 2025 and ASE 2025—specifically CodeRAG, AlignCoder, and SaraCoder—have introduced highly specialized mechanisms to align retrieval intent and enforce context diversity.

CodeRAG: Multi-Path Integration and BestFit Reranking

Presented at EMNLP 2025, CodeRAG confronts the inherent vulnerabilities of relying on a singular retrieval modality.[27] To guarantee the extraction of necessary knowledge, CodeRAG orchestrates a tripartite multi-path retrieval system, simultaneously querying the codebase across three distinct vectors [30]:

1. **Sparse Retrieval (Lexical Relevance):** Utilizing TF-IDF keyword matching to pinpoint syntactically identical code strings and identifier matches.[30]
2. **Dense Retrieval (Semantic Relevance):** Employing highly trained embedding encoders,

such as CodeT5p-220m, to map functional similarity at the function or line granularity, bridging gaps where variable names differ but logic remains consistent.[30]

3. **Dataflow-Guided Retrieval (Structural Relevance):** Constructing and traversing deep dependency graphs to extract structural dependencies and dataflow relationships that dictate how variables mutate across files.[30]

By fusing these three paths, CodeRAG captures complementary relevance signals.[30] However, pooling data from three distinct pipelines threatens to immediately overwhelm the agent's context window. To mitigate this, CodeRAG abandons the naive "last k-lines" prompt formulation.[31] Instead, it employs a sophisticated log probability-guided probing mechanism to construct optimal initial queries, selecting only the code chunks with the highest cumulative predictive scores to serve as the search foundation.[28]

The core architectural innovation of CodeRAG is its solution to retriever-LLM misalignment via "BestFit" reranking.[27] Traditional retrievers rank code by mathematical similarity; CodeRAG recognizes that similarity does not equal utility. BestFit introduces an LLM reranker that directly evaluates the retrieved code pieces for their actual generative *necessity* in completing the specific task.[30] To ensure this LLM-driven reranking remains computationally tractable and avoids massive token expenditure, BestFit utilizes a sliding window and heap sort mechanism. This bounds the LLM comparison operations, allowing the algorithm to scale efficiently at $O(N \log N)$ complexity.[31]

Because querying a massive frontier LLM for continuous reranking in a production environment is financially prohibitive, the CodeRAG framework distills these sophisticated LLM reranking preferences into a significantly smaller, highly efficient local model.[30] This distillation allows production deployments to benefit from LLM-quality ranking precision at a fraction of the inference cost and token usage, resulting in consistent, state-of-the-art outperformance on ReccEval and CrossCodeEval benchmarks.[27]

AlignCoder: Reinforcement Learning for Query Enhancement

A primary catalyst for wasted tokens in codebase retrieval is the profound semantic gap between a developer's prompt (or the immediate local code context) and the optimal target code buried in the repository.[25] If an agent's context window implicitly searches for a "database connection handler," but the repository's strict internal nomenclature refers to it as an SQL_Session_Factory, standard semantic and lexical retrievers will fail, flooding the context window with irrelevant matches and squandering the token budget.

The AlignCoder framework (ASE 2025) directly mitigates this semantic gap through a Query Enhancement mechanism structurally paired with Reinforcement Learning (RL).[25] When faced with an incomplete code block or a query, AlignCoder forces the LLM to autonomously generate multiple *candidate completions* based strictly on the limited local context.[25] While these initial generative guesses are highly likely to be structurally incorrect due to missing

cross-file dependencies, they serve a critical purpose: they naturally bridge the semantic gap by generating tokens, function hypotheses, and variable names that statistically mirror the target code's hidden nomenclature.[25]

These LLM-generated candidate completions are then aggregated to construct an immensely enriched query. Concurrently, an AlignRetriever module is trained utilizing reinforcement learning methodologies.[25] This specialized retriever learns to leverage the inferential information embedded within the enhanced queries, effectively learning how to map the LLM's educated "guesses" directly to the mathematically correct code snippets.[25]

Rigorous ablation studies and benchmark testing confirm the power of this alignment. Across five different backbone code LLMs on the CrossCodeEval benchmark, AlignCoder demonstrated a staggering 18.1% improvement in Exact Match (EM) scores.[25] Furthermore, removing either the query enhancement module or the RL retriever precipitated a severe degradation in EM metrics (ranging from a 7% to 17% drop), proving that mathematically aligning the retrieval intent with the generation target is absolutely essential for optimal token utilization and repository comprehension.[35]

SaraCoder: Maximizing Diversity via Hierarchical Feature Optimization

Even when sophisticated retrievers successfully locate relevant code, they frequently return arrays of data suffering from extreme homogenization and information redundancy. Within large enterprise codebases, utility functions, class interfaces, and boilerplate configurations are heavily duplicated with only microscopic variations. A standard top-k RAG system will blindly retrieve the top five mathematical matches, all of which might be functionally identical. This effectively wastes 80% of the token payload on redundant information, starving the LLM of the diverse context required to solve complex problems.[26]

The SaraCoder framework was explicitly engineered to address this flaw as a resource-optimized retrieval system.[26] It actively combats redundancy and maximizes information diversity and representativeness strictly within a constrained context limit.[26] The core engine driving this efficiency is the Hierarchical Feature Optimization module, which sequentially refines candidate code through multiple rigorous filtering layers before token expenditure occurs [26]:

1. **Semantic Alignment Distillation:** Extracts deep, underlying semantic relationships from the code to assess its true programmatic utility, discarding matches that rely on misleading superficial string similarity.[26]
2. **Redundancy-Aware Pruning:** Actively scans the initial retrieval pool to identify exact duplicates and near-clones, aggressively stripping them out of the pipeline.[26]
3. **Graph-Based Structural Assessment:** Implements a novel graph-based metric that calculates the topological importance of the retrieved code. It assesses how code fragments connect across the repository's dependency graph, weighing edits by their

structural gravity (e.g., modifying a core foundational utility versus a disconnected leaf script).[26]

4. **Diversity-Driven Reranking:** Reorders the final, pruned candidate list to ensure that the LLM is presented with the broadest possible spectrum of unique, structurally distinct contextual clues, resolving the homogenization problem.[26]

Furthermore, to prevent the LLM from hallucinating variable definitions, SaraCoder integrates an External-Aware Identifier Disambiguator.[26] This module automatically resolves cross-file symbol ambiguity by running rapid dependency analysis prior to code generation.[26] Evaluated against challenging benchmarks, SaraCoder proves that systematically curating the context window for maximum diverse signal and minimum redundant noise sets a new benchmark for building highly accurate, resource-optimized repository-level systems.[26]

Externalized State Management: Memory Pointers and Native File Systems

As LLMs have evolved from single-prompt query engines into long-running, autonomous software engineering agents, the foundational strategy of loading all required context directly into the prompt string has proven entirely unsustainable. State-of-the-art research has increasingly shifted toward externalizing the agent's state management. This paradigm allows the model to manipulate massive datasets dynamically without paying the exorbitant token cost of observing the raw data continuously in its context window.

Solving Context Window Overflow via Memory Pointers

In dynamic, knowledge-intensive computational fields such as Materials Science, systems engineering, and complex codebase compilation, executable tools frequently return outputs that instantaneously shatter the maximum bounds of any LLM context window.[40] For instance, a static analysis mapping tool might return a multi-gigabyte dependency graph, or an electronic grid structure simulation might yield a 3D matrix containing millions of float32 elements.[41] Traditional architectural responses to this overflow involve harsh truncation or aggressive summarization. Both techniques permanently destroy critical data fidelity, rendering the indivisible data useless and guaranteeing task failure.[40]

To definitively resolve this, a 2025 research framework introduced the concept of "memory pointers" to handle LLM context overflow without data loss.[40] Rather than passing the raw, massive output of a tool directly into the agent's prompt, the surrounding infrastructure intercepts the output and stores it in an external, highly maintained "runtime memory" database.[41] The infrastructure then returns a short identifier—a memory path or pointer string—directly to the LLM's context window.[40]

The LLM is explicitly trained to interact with data at this higher level of conceptual abstraction.[41] It learns to manipulate these references, passing the pointers as exact arguments to

subsequent tool calls. A "mirrored tool" wrapper handles the execution pipeline. When the agent calls a tool and supplies a pointer, an Input Inspection Module immediately resolves the pointer, retrieves the massive, raw data payload from the runtime memory, and forwards it to the original tool for execution.[41] Following execution, a Post-Processing Module catches the resulting massive output, stores it in memory, and hands a brand new, lightweight pointer back to the LLM agent.[41] To present final results to a human user, a dedicated terminal tool retrieves the explicit values from the runtime memory only when absolutely requested.[41]

This pointer-driven methodology preserves 100% of tool functionality and enables seamless integration into complex agentic workflows.[40] By shifting the model's interaction from raw text to memory pointers, the actual token payload remains infinitesimally small. In comparative empirical experiments on real-world applications that previously caused instant context overflow, this pointer-based approach successfully executed the task while consuming approximately seven times fewer tokens than traditional workflows.[40] It transforms previously infeasible, memory-overflowing operations into routine, cost-efficient actions.[40]

Coding Agents as Native Long-Context Processors

Expanding upon the theme of externalizing context, recent research demonstrates that long-context comprehension can be offloaded entirely from the latent, uninterpretable attention mechanisms of the neural network into explicit, executable terminal interactions.[43]

The paradigm outlined in the study "Coding Agents are Effective Long-Context Processors" argues that rather than relying on RAG pipelines to inject semantic vector chunks into the LLM, the entire repository should be treated natively as a navigable file system of documents and nested folders.[43] The coding agent is granted access to a virtual machine and a terminal environment. When tasked with locating highly specific logic across millions of lines of code, the agent does not ask a vector database. Instead, it utilizes its native tool proficiency to write custom Python scripts, performs programmatic aggregations, uses standard command-line tools like grep and find, and saves intermediate analytical results to temporary text files on the local disk.[43]

The agent continuously iterates on its findings, executing scripts and reading only the highly condensed, short outputs.[43] This grants the agent a robust form of multi-hop reasoning that is entirely unbounded by the rigid constraints of a fixed retrieval pipeline.[43] The efficacy of this approach stems from the agent's inherent file system familiarity.[43] By exploring a massive text corpus as a literal directory structure, the agent dynamically adjusts its search strategy based on real-time discoveries.[43]

Empirical benchmarking demonstrates that agents utilizing this explicit, executable interaction model significantly outperform state-of-the-art semantic search RAG models by an average of 17.3% across multiple complex long-context benchmarks.[43] While coding agents incur slightly higher computational costs than ultra-lightweight baselines, they remain highly competitive and deliver substantially superior performance, decisively proving that delegating massive

context processing to the agent's logical execution environment is a highly effective, transparent alternative to blind context window scaling.[43]

Autonomous Auditing: Data-Flow Tracking and Agent Memory

The absolute necessity for extreme token optimization is perhaps most acutely evident in the domain of autonomous security and repository-level code auditing. Reviewing millions of lines of source code to identify subtle logical flaws, unhandled exceptions, or severe security vulnerabilities is an immensely difficult, labor-intensive task.[46] Standard RAG and semantic search approaches frequently fail catastrophically in auditing scenarios because software bugs are rarely localized to a single chunk; they manifest as complex data-flow violations that traverse multiple discrete files and deeply nested function calls.[46] Analyzing these large-scale execution paths naively incurs astronomical token costs and frequently triggers the LLM to hallucinate false-positive vulnerabilities due to context fragmentation.[46]

To solve this scaling limitation, highly specialized LLM-based agents such as RepoAudit (presented at ICML 2025) have been engineered for autonomous, repository-level code exploration.[46] Rather than continuously indexing static text strings or vector embeddings, RepoAudit's internal memory mechanism is precisely engineered to track and map dynamic "data-flow facts" along feasible program paths across the codebase.[46]

As the RepoAudit agent explores the repository on demand, it continuously builds an internal, highly optimized agent memory graph. This graph maps exactly how specific values and variables mutate and move through different execution contexts and paths.[46] This architectural design allows the agent to hold the conceptual, mathematical logic of the codebase in its active memory without needing to hold the literal, token-heavy strings of code in its prompt.[46]

To aggressively mitigate the high rate of false positives endemic to LLM code analysis, RepoAudit incorporates a deterministic, automated validator module.[46] Before the agent is permitted to report a potential bug, this validator explicitly verifies the accuracy of the tracked data-flow facts.[46] Crucially, it runs an automated satisfiability check on the mathematical conditions associated with the flagged execution path.[46] If the path is mathematically unreachable or the bug requires impossible conditions to trigger, the bug report is silently discarded, sparing human developers from reviewing hallucinations.[46]

The synthesis of structured data-flow memory tracking and automated satisfiability verification allows RepoAudit to achieve unprecedented scalability and cost-efficiency. In rigorous benchmark testing across 15 real-world, widely used open-source projects, the agent detected 40 true bugs with a precision rate of 78.43%.[46] Furthermore, when deployed against high-profile modern repositories, RepoAudit successfully identified 185 previously unknown bugs—174 of which were swiftly confirmed or fixed by human maintainers.[46]

The economic efficiency of this token-optimized approach is remarkable. By utilizing its

specialized memory tracking rather than brute-force RAG context stuffing, RepoAudit averages only 0.44 hours of compute time and requires a mere $2.54 in API token costs per audited project—described as costing "less than a cup of coffee" to secure an entire repository.[46]

Benchmarking Frameworks and the 2026 Model Landscape

As the methodologies for code indexing, multi-path retrieval, and token optimization have grown exponentially in sophistication, the metrics by which they are evaluated have necessarily evolved to maintain rigor. Early code generation benchmarks, such as HumanEval and MBPP, focused predominantly on isolated, single-file function completions.[47] These datasets fail entirely to represent the real-world software engineering ecosystem, where completion success is explicitly predicated on accurately leveraging deep, cross-file dependencies.[47]

Advanced Repository-Level Benchmarks

To adequately measure an agent's ability to navigate, retrieve, and synthesize data within a massive repository without token overflow, modern benchmarking has shifted to comprehensive, multi-file frameworks:

- **CrossCodeEval:** A highly diverse, multilingual benchmark explicitly designed so that accurate code completion is impossible without in-depth cross-file contextual understanding.[47] It utilizes a straightforward yet efficient static-analysis-based approach to pinpoint exact scenarios where functions rely heavily on external files. It currently acts as the gold standard for measuring the capability of code retrievers, establishing the baseline upon which models like AlignCoder and GrepRAG prove their superiority.[25]
- **RepoEval & RepoEval-Updated:** Focuses deeply on line-level and API-level completion tasks, extensively evaluating the accuracy of retrieved contexts across massive enterprise codebases. It is heavily utilized to test models like Repoformer and SaraCoder.[25]
- **SWE-Bench Verified:** Evaluates an agent's capability to resolve real-world GitHub issues by autonomously editing multiple files simultaneously, testing the agent's capacity to maintain repository-wide invariants, avoid syntax breakage, and pass integrated test suites without human intervention.[48]

Performance across these rigorous benchmarks is quantified using Exact Match (EM) percentage, Edit Similarity (ES), and Unit Test Pass Rate (UT), allowing researchers to rigorously, mathematically quantify the superiority of advanced optimization methods over baseline naive retrieval.[21]

The March 2026 Model Landscape

The hardware and foundational model ecosystem as of March 2026 reflects a massive industry

pivot toward native agentic execution, tool fluency, and deep token efficiency. The landscape is dominated by foundational models that integrate advanced tool-use directly into their post-training reinforcement learning regimens:

- **GPT-5.4 Unified and Codex Platforms:** Following the strategic unification of their general and coding model lines, OpenAI's flagship models now feature native computer use. The dedicated Codex platform offers a highly specialized environment explicitly trained across 20K parallel coding environments for long-horizon software engineering work, supporting full-repository comprehension and multi-file refactoring in a single context.[49]
- **Claude Sonnet Ecosystem:** Widely recognized across the industry for achieving "Level 4 tool fluency." This architecture excels at natively operating browser environments, managing internal debuggers, and interacting flawlessly with dynamically generated API outputs without requiring excessive prompting tokens.[49]
- **Open-Weight and Local SOTA:** The open-weight ecosystem has largely closed the gap with proprietary giants. Nvidia's Kimi K2.5 and GLM-5 APIs provide highly capable agentic reasoning accessible via platforms like OpenCode.[50] More notably, models like the open-source OpenSeeker (released March 16, 2026) have democratized frontier search agent research.[51] Highly efficient local models like GLM 4.7 Flash (optimized to run on consumer-grade 24GB VRAM hardware) are delivering near-SOTA intelligence with advanced tool handling, proving that localized filtering and execution are economically and practically viable.[52]

The convergence of these highly fluent models with sophisticated local workflow tools—such as the Cursor IDE's proposed local pre-filtering system, which uses lightweight local models to extract AST elements and enrich prompts *before* hitting expensive cloud APIs—represents the final maturation of the token-optimized coding stack.[5]

Synthesis and Strategic Conclusions

The empirical research spanning from early 2024 through the advanced frameworks of 2026 reveals a decisive and irreversible paradigm shift in how LLMs interact with source code. The initial industry instinct to solve large-scale code comprehension by simply expanding the context window to millions of tokens has proven economically inefficient, computationally flawed, and susceptible to severe context pollution. The sheer financial cost of processing vast amounts of extraneous input tokens, coupled with the latency introduced by massive prompts, dictates that the future of AI software engineering lies in extreme token optimization and intelligent state management.

The proven, peer-reviewed methodologies demonstrate that the most effective token optimization strategies do not rely on a single algorithm, but rather on a composite architecture of highly selective, structure-aware, and agent-driven workflows.

First, the baseline representation of code for any retrieval mechanism must transition from flat,

naive text arrays to structurally bound Abstract Syntax Trees. Techniques like the cAST framework definitively prove that preserving semantic boundaries during the chunking phase fundamentally upgrades the baseline intelligence of the RAG system. By ensuring chunks are syntactically valid subtrees, agents cease wasting tokens on broken logic flows and hallucinated scopes.[12] When coupled with hierarchical summarization, agents can traverse codebases conceptually before expending tokens on raw extraction.[19]

Second, the system must employ rigorous, self-evaluating gatekeeping. The Repoformer paradigm establishes that invariant, ubiquitous retrieval is a systemic flaw, not a feature.[21] By fine-tuning models via FIM mechanisms to self-evaluate and abstain from retrieval up to 80% of the time, inference speed is dramatically accelerated by 70%, and token costs are slashed without degrading generative accuracy.[20]

Third, when retrieval is mathematically deemed necessary, the reliance on computationally heavy dense semantic indexing must be re-evaluated and balanced with precise alternatives. Index-free lexical approaches like GrepRAG highlight that exact identifier matching via terminal utilities often outperforms complex vector math in a strict coding environment, bypassing indexing costs entirely.[23] Furthermore, when dense retrieval is utilized, pipelines must incorporate Reinforcement Learning alignments (such as AlignCoder's query enhancement) to bridge semantic gaps [25], and aggressive redundancy pruning (such as SaraCoder's topological graphing) to ensure every token passed to the context window provides maximum informational diversity.[26]

Ultimately, the pinnacle of token optimization is the complete externalization of long-context processing. By equipping autonomous agents with memory pointers and the native ability to navigate repositories as standard file systems using Python scripts and CLI utilities, the LLM ceases to be a static, prompt-processing engine.[41] Instead, it evolves into a dynamic, iterative coordinator. It tracks data-flow facts in dedicated external memory graphs [46] and references multi-gigabyte structures via lightweight, short pointer strings.[40] This allows the agent to maintain absolute cognitive coherence over massive enterprise repositories, audit complex security flaws autonomously, and execute multi-file refactoring while systematically utilizing only a fraction of its available context window. This synthesis of structural parsing, selective abstention, aligned multi-path retrieval, and pointer-based agentic workflow represents the definitive, proven architectural standard for optimizing token usage in modern LLM coding agents.

Cytowane prace

1. Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2510.04905v1
2. Retrieval-Augmented Code Generation: A Survey with Focus ... - arXiv, otwierano: marca 25, 2026, https://arxiv.org/abs/2510.04905
3. Context Engineering - LangChain Blog, otwierano: marca 25, 2026,

https://blog.langchain.com/context-engineering-for-agents/
4. AI Agentic Programming: A Survey of Techniques, Challenges, and Opportunities - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2508.11126v1
5. Idea: Optimize LLM Usage with a Local Filter & Code Caching (reduce token costs), otwierano: marca 25, 2026, https://forum.cursor.com/t/idea-optimize-llm-usage-with-a-local-filter-code-caching-reduce-token-costs/91985
6. What is a context window? - IBM, otwierano: marca 25, 2026, https://www.ibm.com/think/topics/context-window
7. Effective context engineering for AI agents - Anthropic, otwierano: marca 25, 2026, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
8. DeepSoftwareAnalytics/Awesome-Agent4SE - GitHub, otwierano: marca 25, 2026, https://github.com/deepsoftwareanalytics/awesome-agent4se
9. CAST: Enhancing Code Summarization with Hierarchical Splitting and Reconstruction of Abstract Syntax Trees - ACL Anthology, otwierano: marca 25, 2026, https://aclanthology.org/2021.emnlp-main.332/
10. We reduced token usage by 60% using an agentic retrieval protocol. Here's how. - Reddit, otwierano: marca 25, 2026, https://www.reddit.com/r/AI_Agents/comments/1jugj0e/we_reduced_token_usage_by_60_using_an_agentic/
11. Token Optimization Strategies for AI Agents | by Netanel Avraham | Elementor Engineers, otwierano: marca 25, 2026, https://medium.com/elementor-engineers/optimizing-token-usage-in-agent-based-assistants-ffd1822ece9c
12. cAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2506.15655v1
13. CAST: Enhancing Code Retrieval-Augmented Generation with Structural Chunking via Abstract Syntax Tree - ACL Anthology, otwierano: marca 25, 2026, https://aclanthology.org/2025.findings-emnlp.430.pdf
14. Building a Knowledge Assistant over Code | Databricks Blog, otwierano: marca 25, 2026, https://www.databricks.com/blog/building-knowledge-assistant-over-code
15. A Survey on Code Generation with LLM-based Agents - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2508.00083v1
16. An AST-guided LLM Approach for SVRF Code Synthesis - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2507.00352v1
17. Enhancing LLM Code Generation with RAG and AST-Based Chunking | by VXRL - Medium, otwierano: marca 25, 2026, https://vxrl.medium.com/enhancing-llm-code-generation-with-rag-and-ast-based-chunking-5b81902ae9fc
18. Explore whether 'smart code search' (AST, stack graphs, scope graphs) would improve performance beyond the current find/grep based approach. · Issue #38 ·

SWE-agent/SWE-agent - GitHub, otwierano: marca 25, 2026, https://github.com/princeton-nlp/SWE-agent/issues/38
19. (PDF) Repository-Level Code Understanding by LLMs via ..., otwierano: marca 25, 2026, https://www.researchgate.net/publication/391739021_Repository-Level_Code_Understanding_by_LLMs_via_Hierarchical_Summarization_Improving_Code_Search_and_Bug_Localization
20. Repoformer: Selective Retrieval for Repository-Level Code Completion - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2403.10059v2
21. Enhancing repository-level code completion with selective retrieval - Amazon Science, otwierano: marca 25, 2026, https://www.amazon.science/blog/enhancing-repository-level-code-completion-with-selective-retrieval
22. Repoformer: Selective Retrieval for Repository-Level Code Completion, otwierano: marca 25, 2026, https://icml.cc/media/icml-2024/Slides/33155.pdf
23. GrepRAG: An Empirical Study and Optimization of Grep-Like Retrieval for Code Completion, otwierano: marca 25, 2026, https://www.researchgate.net/publication/400340391_GrepRAG_An_Empirical_Study_and_Optimization_of_Grep-Like_Retrieval_for_Code_Completion
24. Shows GrepRAG Unlocks Repository-Level Code Completion With Index-Free Retrieval For LLMs - Quantum Zeitgeist, otwierano: marca 25, 2026, https://quantumzeitgeist.com/shows-greprag-unlocks-repository-level/
25. AlignCoder: Aligning Retrieval with Target Intent for Repository-Level Code Completion, otwierano: marca 25, 2026, https://arxiv.org/html/2601.19697v1
26. SaraCoder: Orchestrating Semantic and Structural Cues for Resource-Optimized Repository-Level Code Completion | OpenReview, otwierano: marca 25, 2026, https://openreview.net/forum?id=Fi1V7OgNqJ
27. CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion - ACL Anthology, otwierano: marca 25, 2026, https://aclanthology.org/2025.emnlp-main.1187/
28. CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion - Semantic Scholar, otwierano: marca 25, 2026, https://www.semanticscholar.org/paper/CodeRAG%3A-Finding-Relevant-and-Necessary-Knowledge-Zhang-Ding/ba9db9ccb8e3677830da0c55b4a0b6c97f6bbd29
29. [2509.16112] CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion - arXiv, otwierano: marca 25, 2026, https://arxiv.org/abs/2509.16112
30. CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion | alphaXiv, otwierano: marca 25, 2026, https://www.alphaxiv.org/overview/2509.16112
31. CodeRAG: Retrieval-Augmented Code Synthesis - Emergent Mind, otwierano: marca 25, 2026, https://www.emergentmind.com/topics/coderag
32. CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented

Repository-Level Code Completion - ACL Anthology, otwierano: marca 25, 2026, https://aclanthology.org/2025.emnlp-main.1187.pdf
33. [2601.19697] AlignCoder: Aligning Retrieval with Target Intent for Repository-Level Code Completion - arXiv, otwierano: marca 25, 2026, https://arxiv.org/abs/2601.19697
34. AlignCoder: Aligning Retrieval with Target Intent for Repository-Level Code Completion (ASE 2025 - Research Papers) - conf.researchr.org, otwierano: marca 25, 2026, https://conf.researchr.org/details/ase-2025/ase-2025-papers/34/AlignCoder-Aligning-Retrieval-with-Target-Intent-for-Repository-Level-Code-Completio
35. AlignCoder: Unified Alignment in Code & Genomics - Emergent Mind, otwierano: marca 25, 2026, https://www.emergentmind.com/topics/aligncoder
36. AlignCoder: Aligning Retrieval with Target Intent for Repository-Level Code Completion | Request PDF - ResearchGate, otwierano: marca 25, 2026, https://www.researchgate.net/publication/400119369_AlignCoder_Aligning_Retrieval_with_Target_Intent_for_Repository-Level_Code_Completion
37. SARACODER: ORCHESTRATING SEMANTIC AND STRUCTURAL CUES FOR RESOURCE-OPTIMIZED REPOSITORY-LEVEL CODE COMPLETION - OpenReview, otwierano: marca 25, 2026, https://openreview.net/pdf/f2c1248cb3ac8440f4771abf1bce1d744b4b2498.pdf
38. SaraCoder: Orchestrating Semantic and Structural Cues for Profit-Oriented Repository-Level Code Completion - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2508.10068v1
39. [2508.10068] SaraCoder: Orchestrating Semantic and Structural Cues for Resource-Optimized Repository-Level Code Completion - arXiv, otwierano: marca 25, 2026, https://arxiv.org/abs/2508.10068
40. [2511.22729] Solving Context Window Overflow in AI Agents - arXiv, otwierano: marca 25, 2026, https://arxiv.org/abs/2511.22729
41. Solving Context Window Overflow in AI Agents - arXiv, otwierano: marca 25, 2026, https://www.arxiv.org/pdf/2511.22729
42. (PDF) Solving Context Window Overflow in AI Agents - ResearchGate, otwierano: marca 25, 2026, https://www.researchgate.net/publication/398136030_Solving_Context_Window_Overflow_in_AI_Agents
43. Coding Agents are Effective Long-Context Processors - arXiv, otwierano: marca 25, 2026, https://arxiv.org/html/2603.20432v1
44. Making Myths Material: What Does It Mean to be REF-able? - ResearchGate, otwierano: marca 25, 2026, https://www.researchgate.net/publication/400378028_Making_Myths_Material_What_Does_It_Mean_to_be_REF-able
45. Selection of experts for scientific and technical expertise based on semantic search | Request PDF - ResearchGate, otwierano: marca 25, 2026, https://www.researchgate.net/publication/353396113_Selection_of_experts_for_scientific_and_technical_expertise_based_on_semantic_search
46. ICML Poster RepoAudit: An Autonomous LLM-Agent for Repository ..., otwierano:

marca 25, 2026, https://icml.cc/virtual/2025/poster/45170
47. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion, otwierano: marca 25, 2026, https://openreview.net/forum?id=wgDcbBMSfh
48. Repository-Level Code Generation - Emergent Mind, otwierano: marca 25, 2026, https://www.emergentmind.com/topics/repository-level-code-generation
49. Most powerful LLMs (Large Language Models) in 2026 - Codingscape, otwierano: marca 25, 2026, https://codingscape.com/blog/most-powerful-llms-large-language-models
50. AI Coding Assistants: Value, Workflow, and Tradeoffs (March 2026) | by Teng Fone, otwierano: marca 25, 2026, https://medium.com/@tengfone/ai-coding-assistants-value-workflow-and-tradeoffs-march-2026-a830a7bc22f2
51. Trending Papers - Hugging Face, otwierano: marca 25, 2026, https://huggingface.co/papers/trending
52. How close are open-weight models to "SOTA"? My honest take as of today, benchmarks be damned. : r/LocalLLaMA - Reddit, otwierano: marca 25, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1qrsy4q/how_close_are_openweight_models_to_sota_my_honest/